Plant Nursery Simulator - Final Project Report

LINK OF THE REPORT: Plant Nursery Simulator - Project Report

ITERATOR INNOVATORS

GROUP MEMBERS:

Gift Mohuba	**u23535527**
Christopher Adolph	**u23535548**
Lusanda Mtembu	**u23602016**
Tiego Mokwena	**u22496336**
Lufuno Mphagi	**u22501445**

Research Brief

Introduction to Plant Nursery Management

A plant nursery is a complex ecosystem that requires careful management of biological, environmental, and commercial factors. Our research focused on understanding the real-world operations of nurseries to inform our software design decisions.

Key Research Findings

Plant Classification and Care Requirements:

- Flowers (Roses, Lavender): Require regular watering, specific sunlight exposure, and seasonal care. Bloom cycles vary by species.
- Trees (Baobab, Oak): Long growth cycles, require structural pruning, and have deep root systems needing specialized soil.
- Vegetables (Tomatoes, Carrots): Have defined harvest times, specific nutrient requirements, and seasonal planting schedules.
- Shrubs (Boxwood, Hydrangeas): Need regular trimming, moderate water, and can be shaped for decorative purposes.

Plant Life Cycle Stages:

1. Seedling: High vulnerability, requires constant moisture and protection
2. Growing: Rapid development, needs balanced nutrients and space
3. Mature: Stable state, requires maintenance care
4. Wilting: Stress response, needs intervention for recovery
5. Dead: End of life cycle, requires removal and replacement

Environmental Factors:

- Temperature fluctuations affect growth rates
- Humidity levels impact water requirements
- Seasonal changes dictate care routines and sales patterns

Design Influences from Research

State Pattern Implementation:
The plant life cycle research directly influenced our State pattern design. Each state

(SeedlingState, GrowingState, MatureState, WiltingState, DeadState) encapsulates behavior specific to real botanical growth stages.

Strategy Pattern for Care Routines:
Different plant species require specialized care approaches:

- LowMaintenanceCare: For drought-resistant plants like cacti
- HighMaintenanceCare: For delicate flowers like orchids
- SeasonalCare: For plants sensitive to seasonal changes

Inventory Management:
Real nurseries track:

- Stock levels and reorder points
- Plant maturity for sales readiness
- Supplier relationships and restocking cycles

Assumptions and Definitions

Assumptions Made:

1. Plant growth can be simulated in daily cycles
2. Environmental factors can be modeled as numerical values
3. Staff members follow assigned routines consistently
4. Customer behavior follows predictable patterns
5. Plant health can be quantified numerically

Key Definitions:

- Plant Health: Numerical value (0-100) representing overall vitality
- Maturity: Stage when plant is ready for sale
- Care Strategy: Algorithm defining how a plant should be maintained
- Inventory Item: Stock-keeping unit combining plant data with quantity

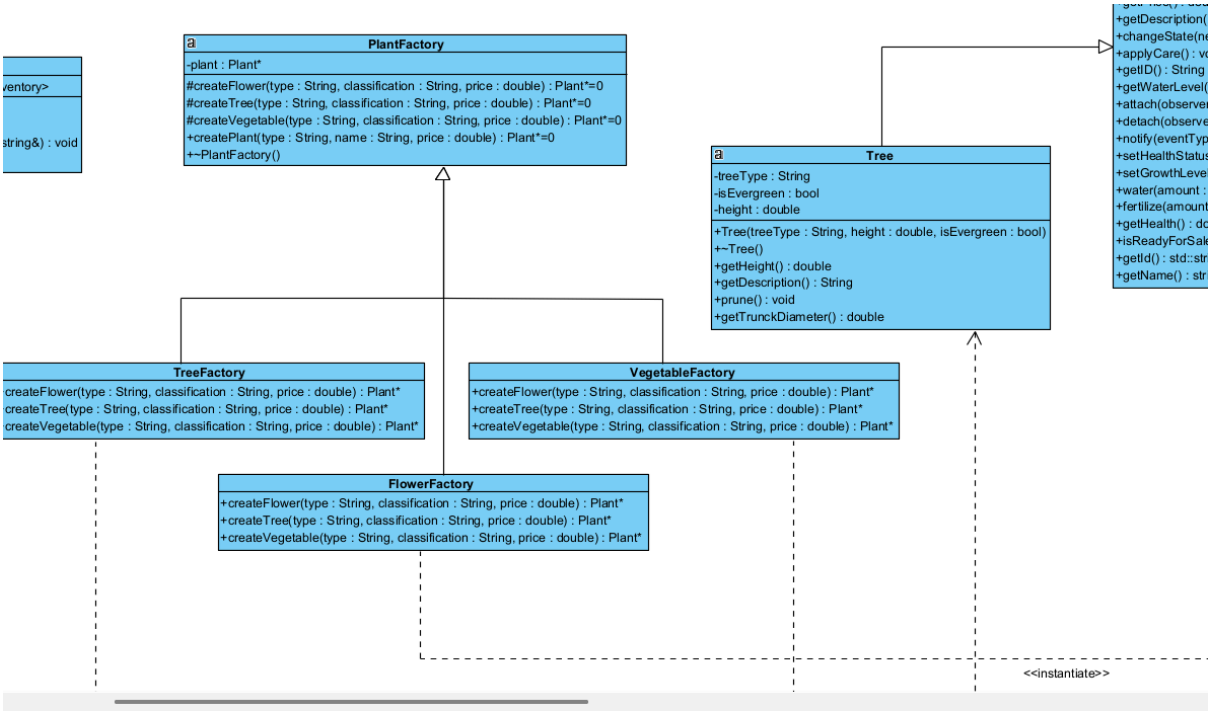
Pattern Implementation Justification

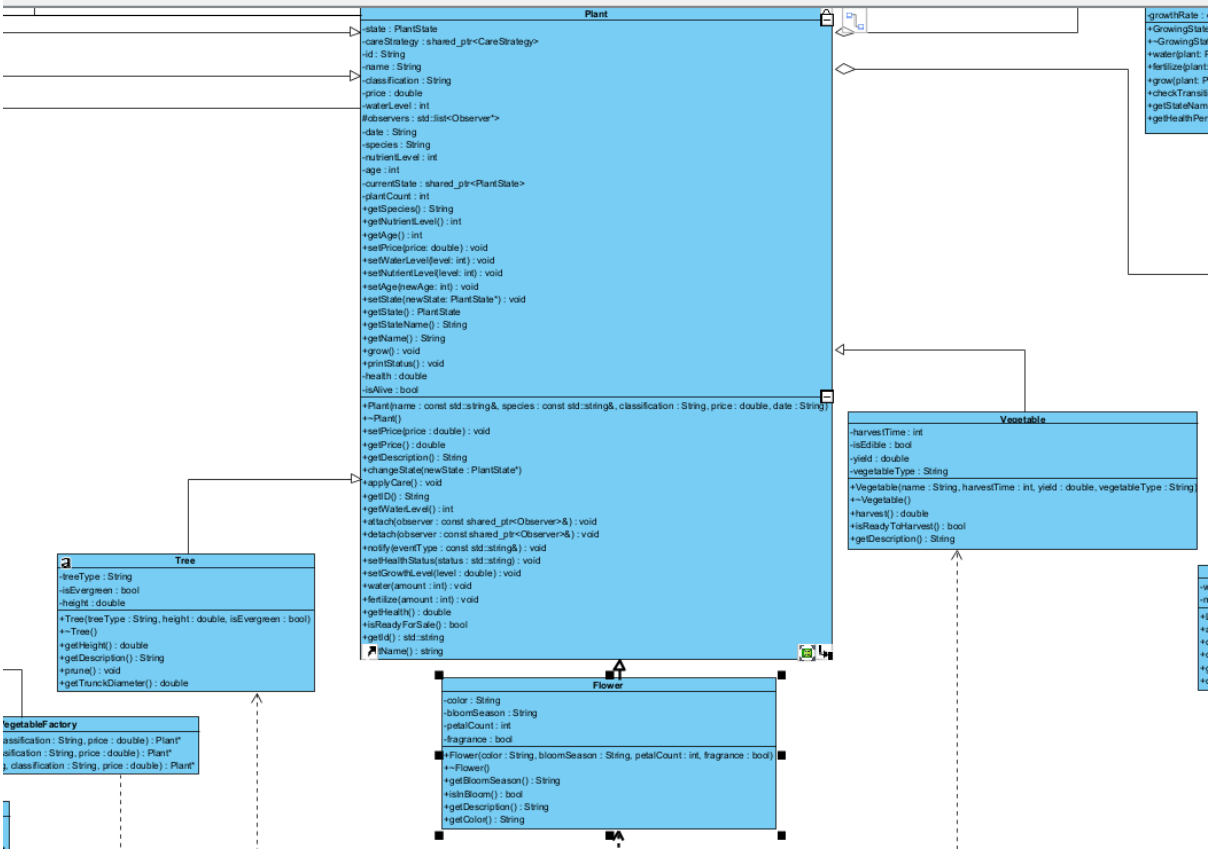
1. Factory Method Pattern

Implementation: PlantFactory, FlowerFactory, TreeFactory, VegetableFactory
Purpose: Encapsulate plant creation logic and allow runtime selection of plant types
Justification: Different plant types require different initialization parameters and behaviors. Factory Method enables creating specific plant subtypes without coupling

client code to concrete classes.

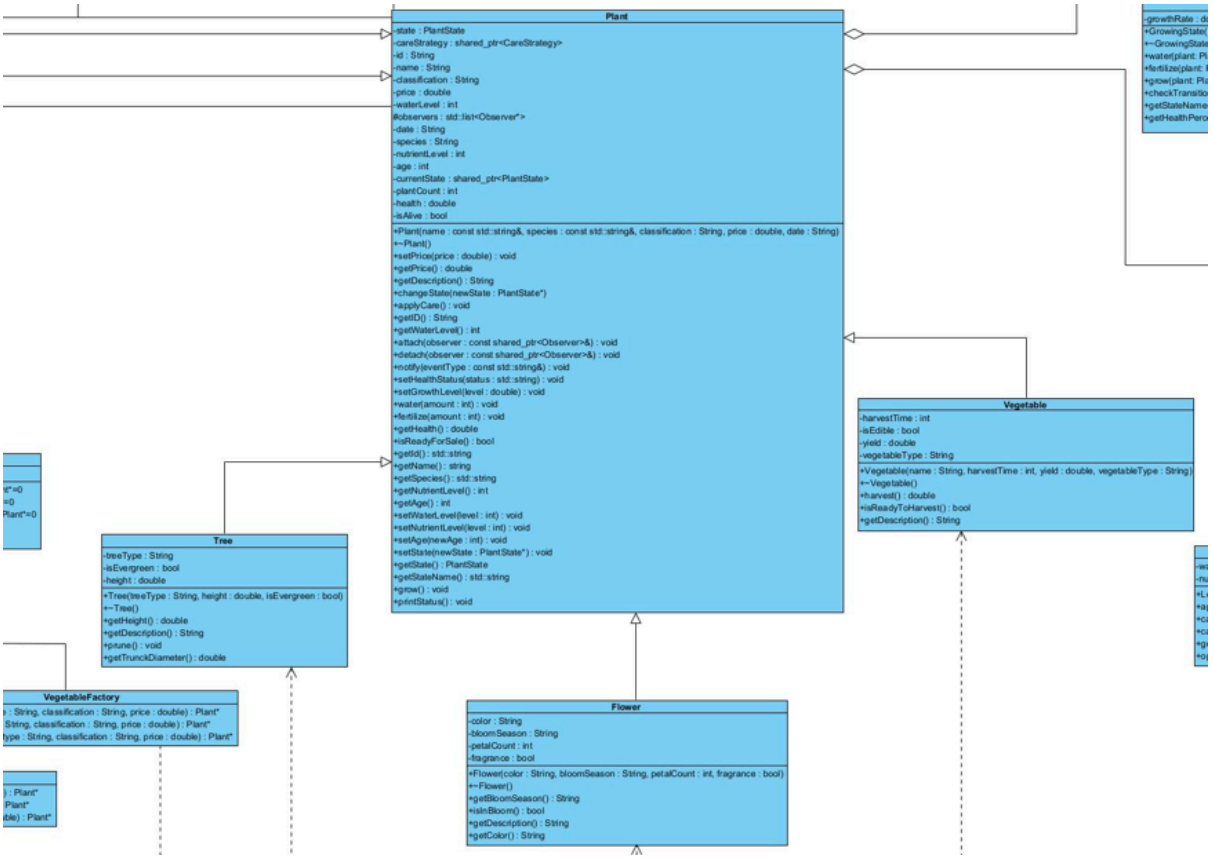
Functional Requirement: FR1 - Create different types of plants





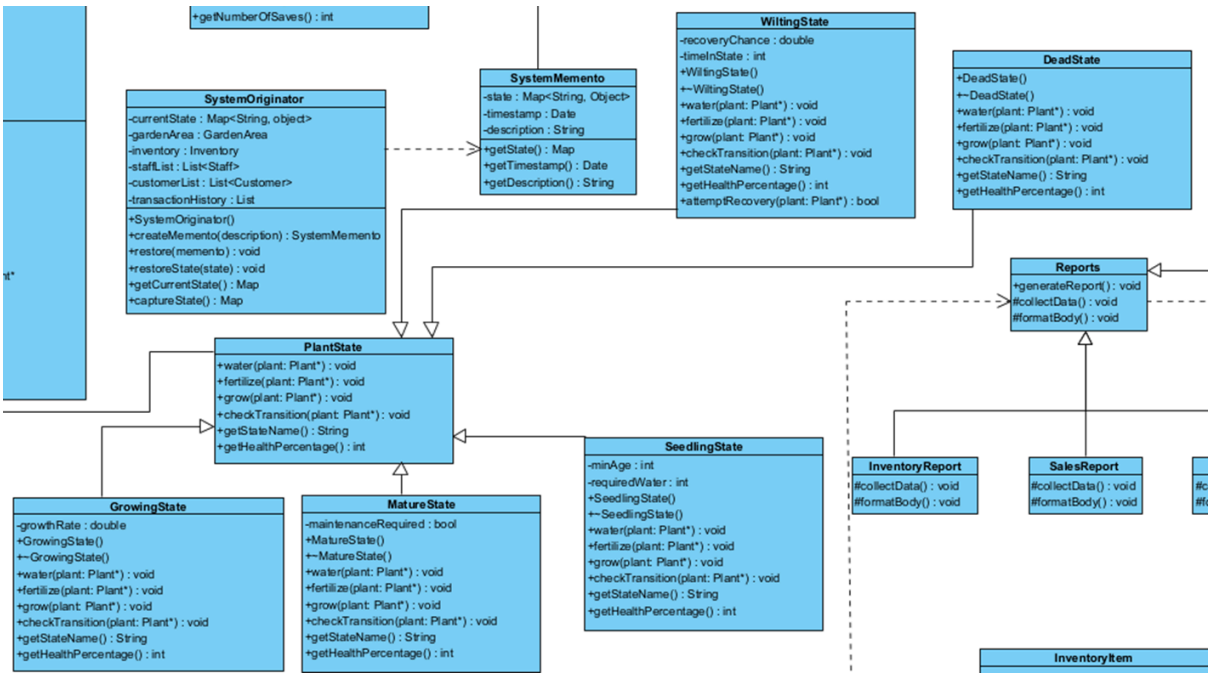
2. Abstract Factory Pattern

Implementation: PlantAbstractFactory, FlowerPlantFactory, TreePlantFactory
Purpose: Create families of related objects (plants with their care kits and soil)
Justification: Ensures compatibility between plants and their accessories. A flower plant should come with flower-appropriate soil and care instructions.
Functional Requirement: FR1 - Create different types of plants with complete care packages



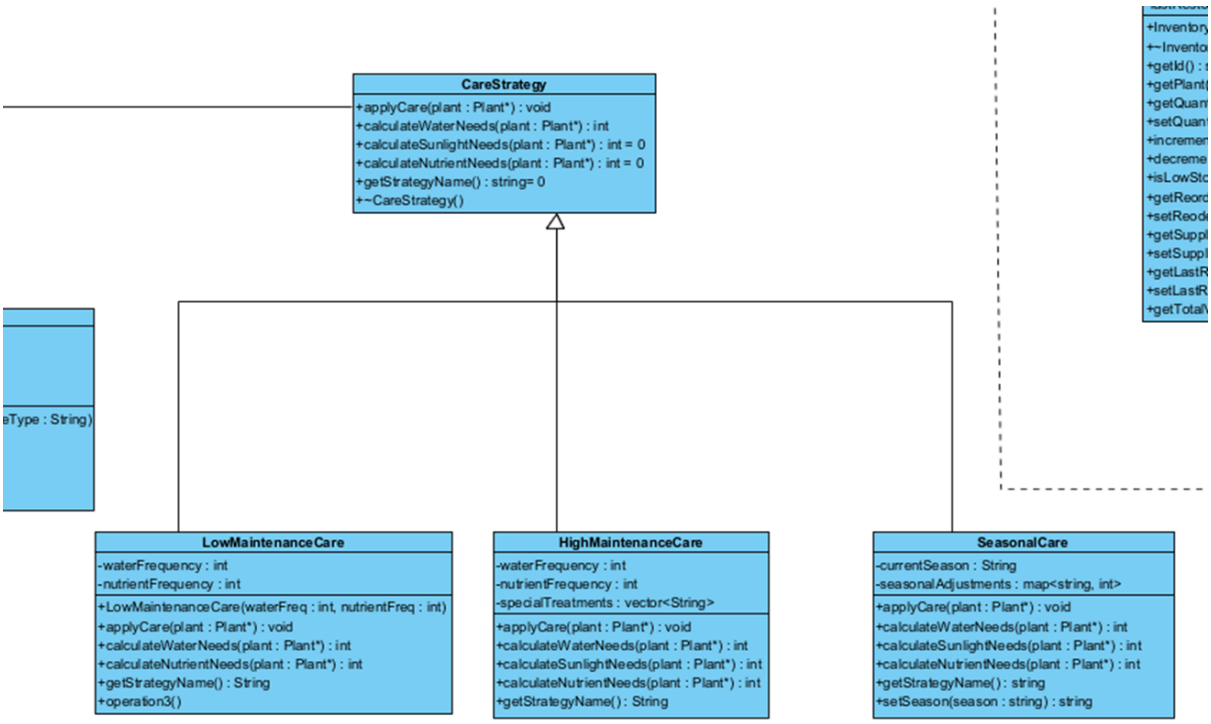
3. State Pattern

Implementation:PlantState, SeedlingState, GrowingState, MatureState, WiltingState, DeadState
Purpose: Manage plant life cycle states and state-specific behavior
Justification: Plants naturally transition through life stages with different care requirements. State pattern localizes state-specific behavior and makes transitions explicit.
Functional Requirement: FR2 - Track plant health states and life cycles



4. Strategy Pattern

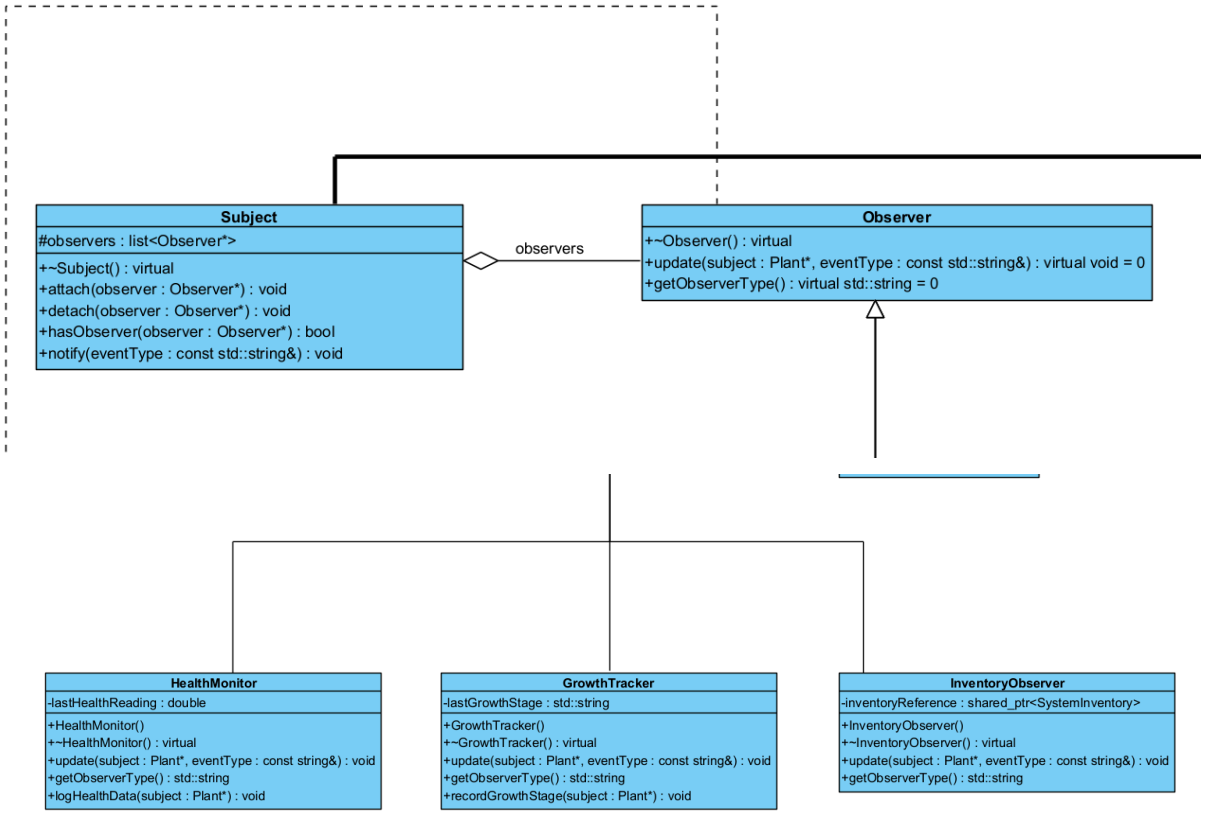
Implementation:CareStrategy,LowMaintenanceCare,HighMaintenanceCare,SeasonalCare
Purpose: Define interchangeable care algorithms for different plant types
Justification: Plants have varying care requirements. Strategy pattern allows runtime selection of care routines without modifying plant classes.
Functional Requirement: FR3 - Apply different care strategies based on plant types



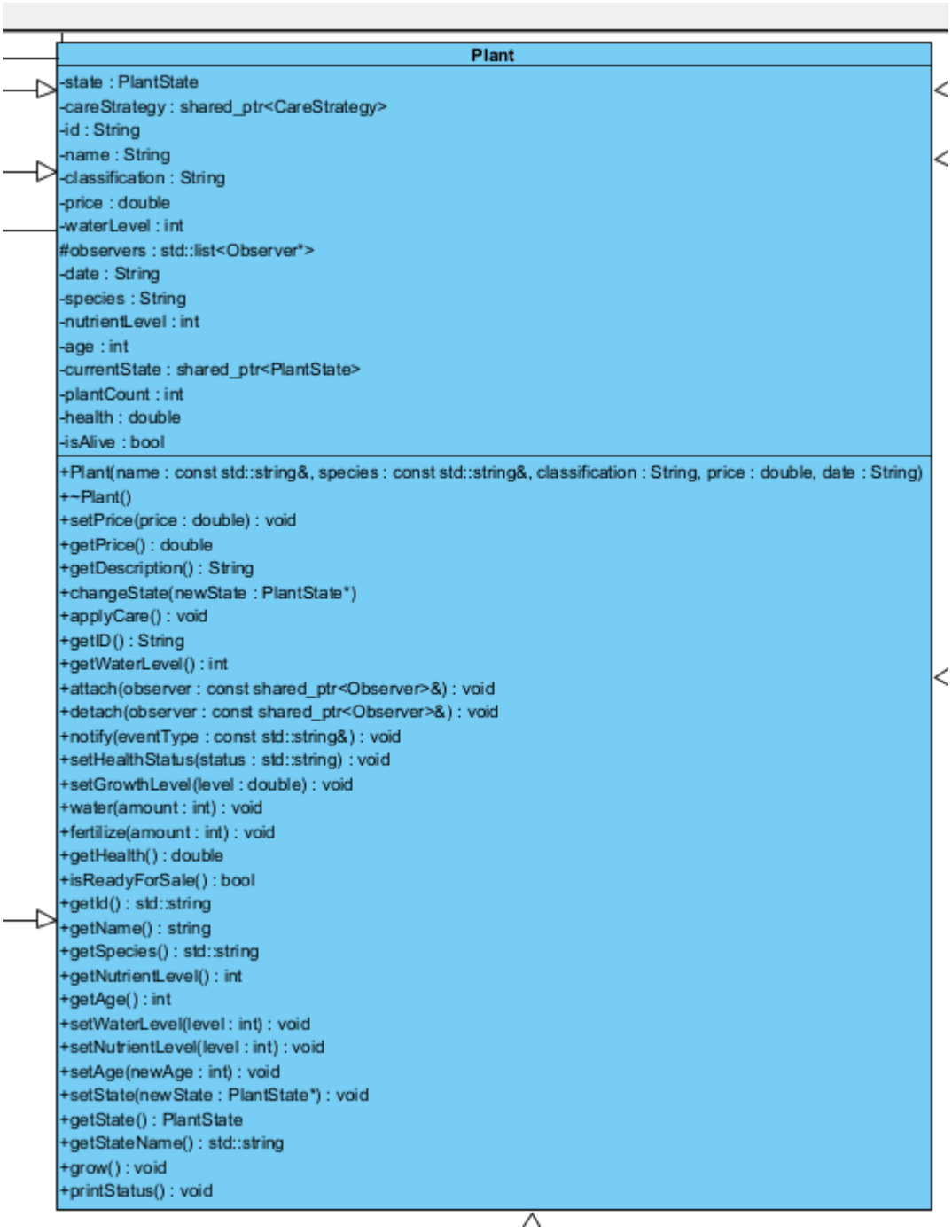
5. Observer Pattern

Implementation:Observer, Subject, HealthMonitor, GrowthTracker, InventoryObserver
Purpose: Implement publish-subscribe mechanism for plant state changes
Justification: Multiple systems need to react to plant changes (inventory, health monitoring, growth tracking). Observer decouples these systems from plant implementation.
Functional Requirement: FR4 - Monitor plant health and notify relevant systems

Observer & Subject at the top and the Concrete Observers at the bottom:

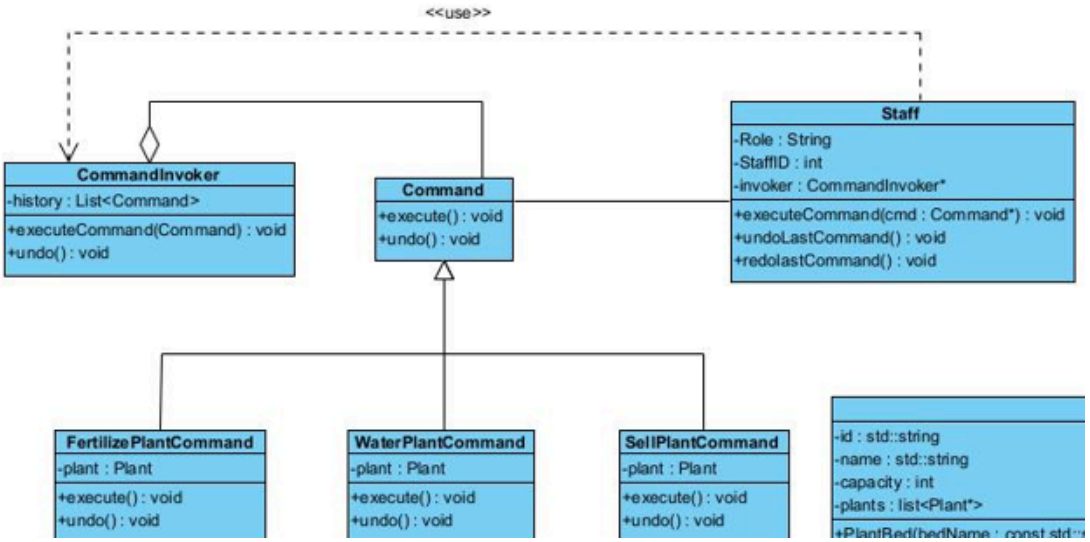


Concrete Subject:



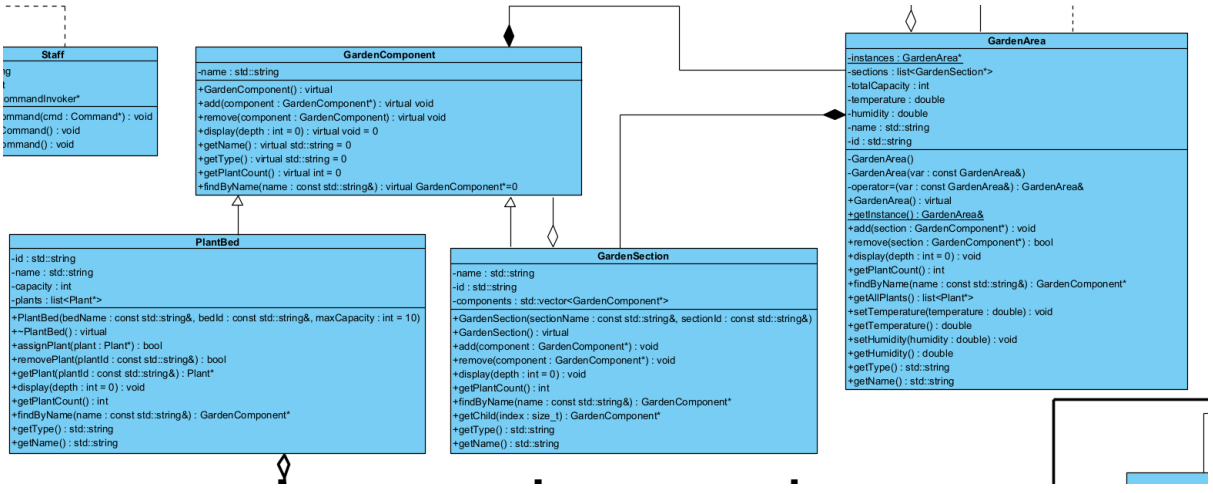
6. Command Pattern

Implementation:Command, WaterPlantCommand, FertilizePlantCommand, SellPlantCommand
Purpose: Encapsulate staff operations as objects
Justification: Staff actions need to be logged, undone, and queued. Command pattern provides transaction-like behavior for nursery operations.
Functional Requirement: FR5 - Execute and track staff operations



7. Composite Pattern

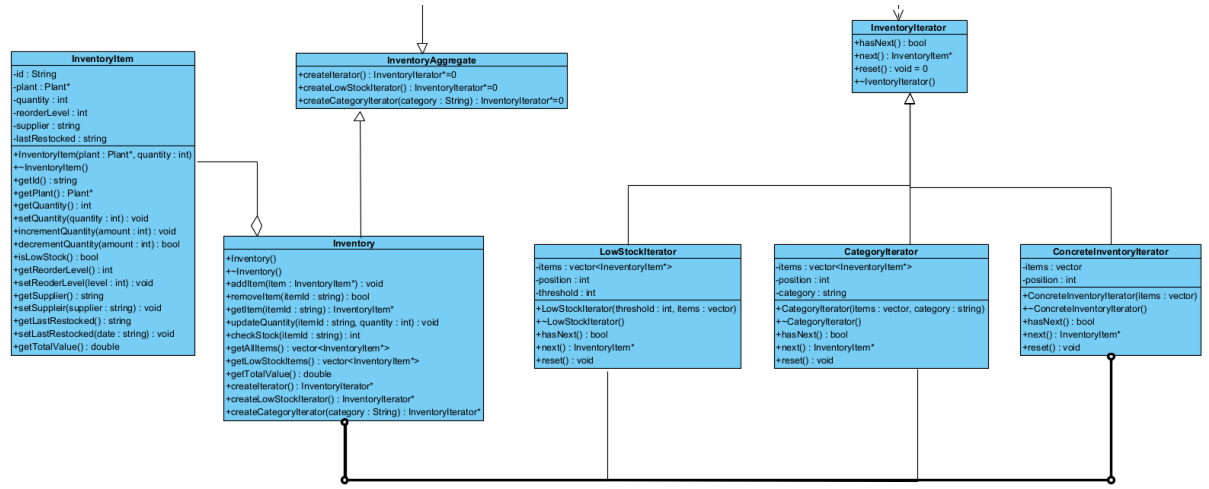
Implementation:GardenComponent, GardenSection, PlantBed
Purpose: Treat individual plants and plant groups uniformly
Justification: Garden organization is naturally hierarchical. Composite enables recursive operations across the entire garden structure.
Functional Requirement: FR6 - Organize plants in hierarchical garden sections

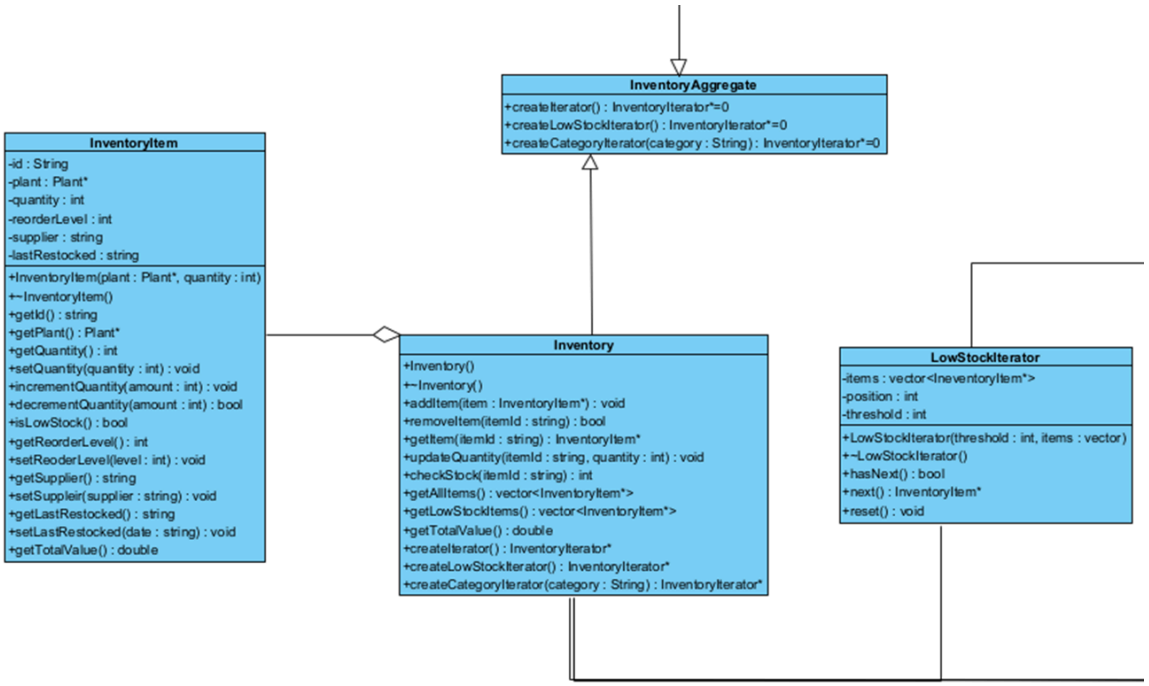


8. Iterator Pattern

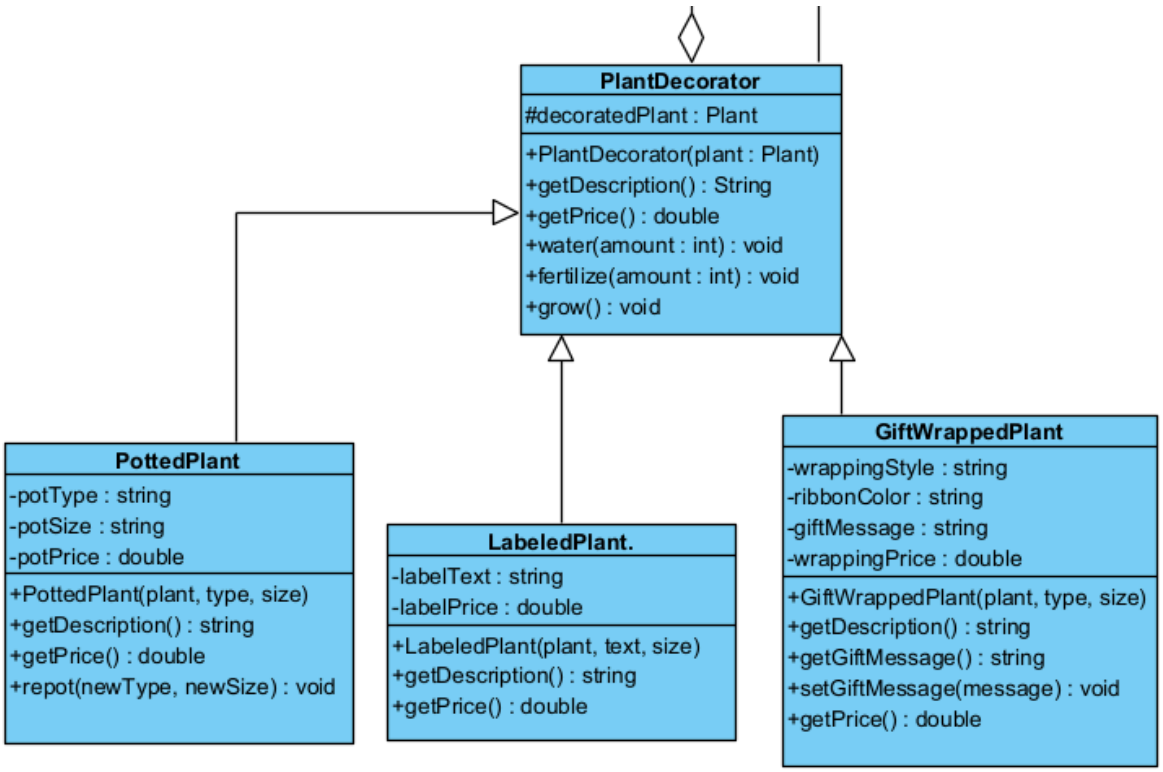
Implementation:InventoryIterator, ConcreteInventoryIterator, LowStockIterator, CategoryIterator
Purpose: Provide multiple ways to traverse inventory collections
Justification: Different contexts require different traversal methods (all items, low stock, by category). Iterator encapsulates traversal logic.

Functional Requirement: FR7 - Generate inventory reports with different filtering





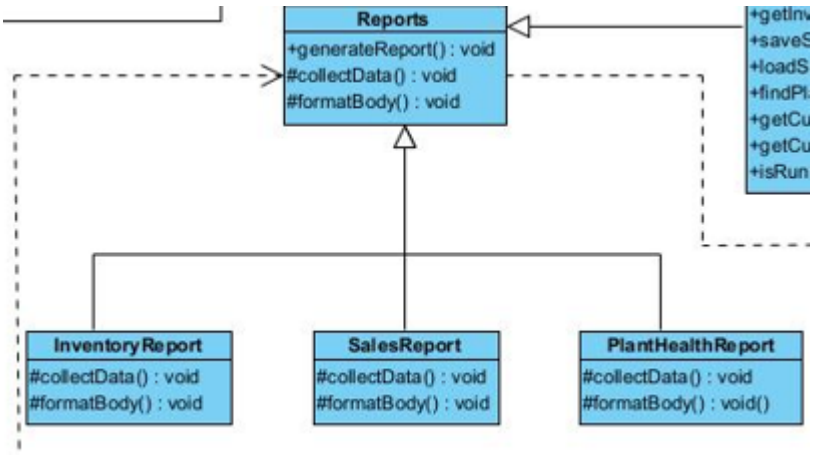
9. Decorator Pattern



Implementation:PlantDecorator, PottedPlant, LabeledPlant, GiftWrappedPlant
Purpose: Dynamically add enhancements to plants
Justification: Customers often request additions like decorative pots or gift wrapping. Decorator enables adding features without modifying plant hierarchy.
Functional Requirement: FR8 - Support plant customization for customers

10. Template Method Pattern

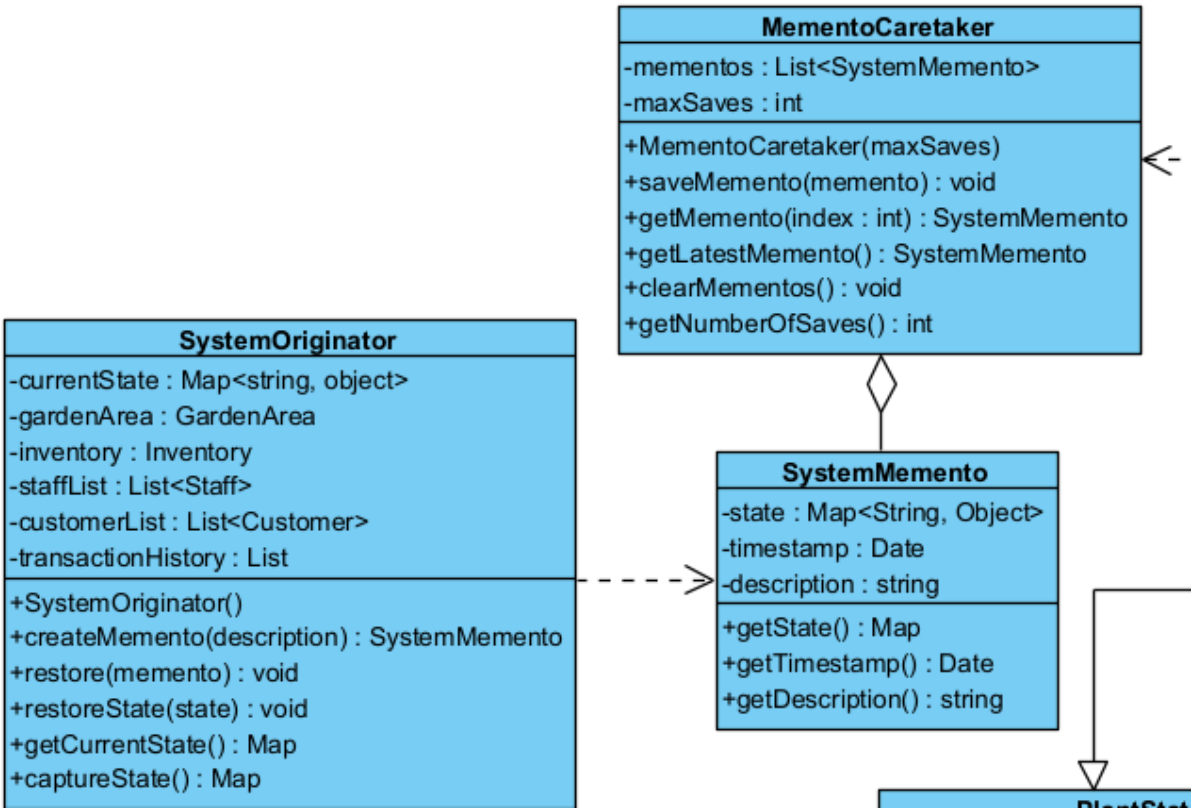
Implementation: Report, InventoryReport, SalesReport, PlantHealthReport
Purpose: Define report generation algorithm skeleton
Justification: All reports follow similar structure but with different data. Template Method eliminates code duplication while allowing customization.
Functional Requirement: FR9 - Generate various types of reports



11. Facade Pattern

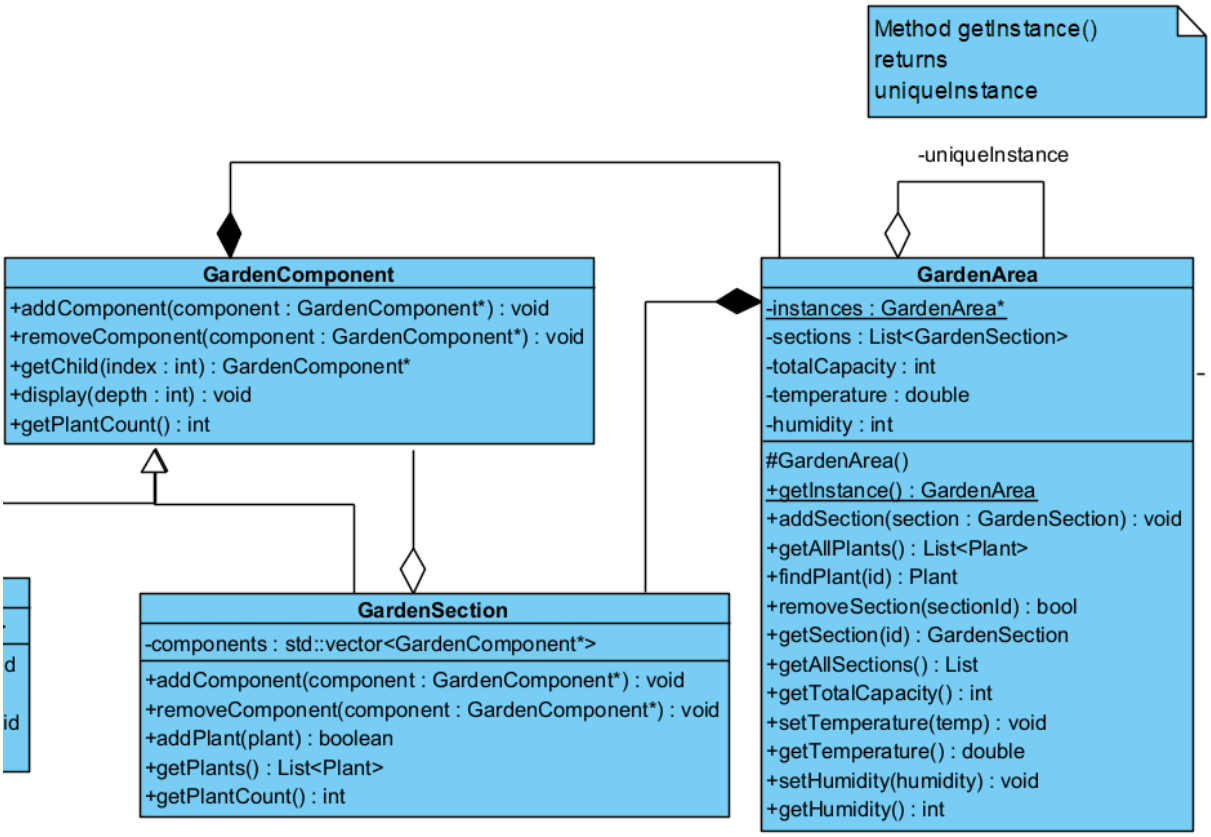
Implementation: SimulationFacade
Purpose: Provide simplified interface to complex subsystem interactions
Justification: Nursery simulation involves multiple complex subsystems. Facade hides this complexity from client code.
Functional Requirement: FR10 - Provide easy-to-use simulation interface

12. Memento Pattern



Implementation: SystemMemento, SystemOriginator, MementoCaretaker
Purpose: Capture and restore system state
Justification: Simulation progress needs to be saved and loaded. Memento preserves encapsulation while enabling state persistence.
Functional Requirement: FR11 - Save and load simulation state

13. Singleton Pattern



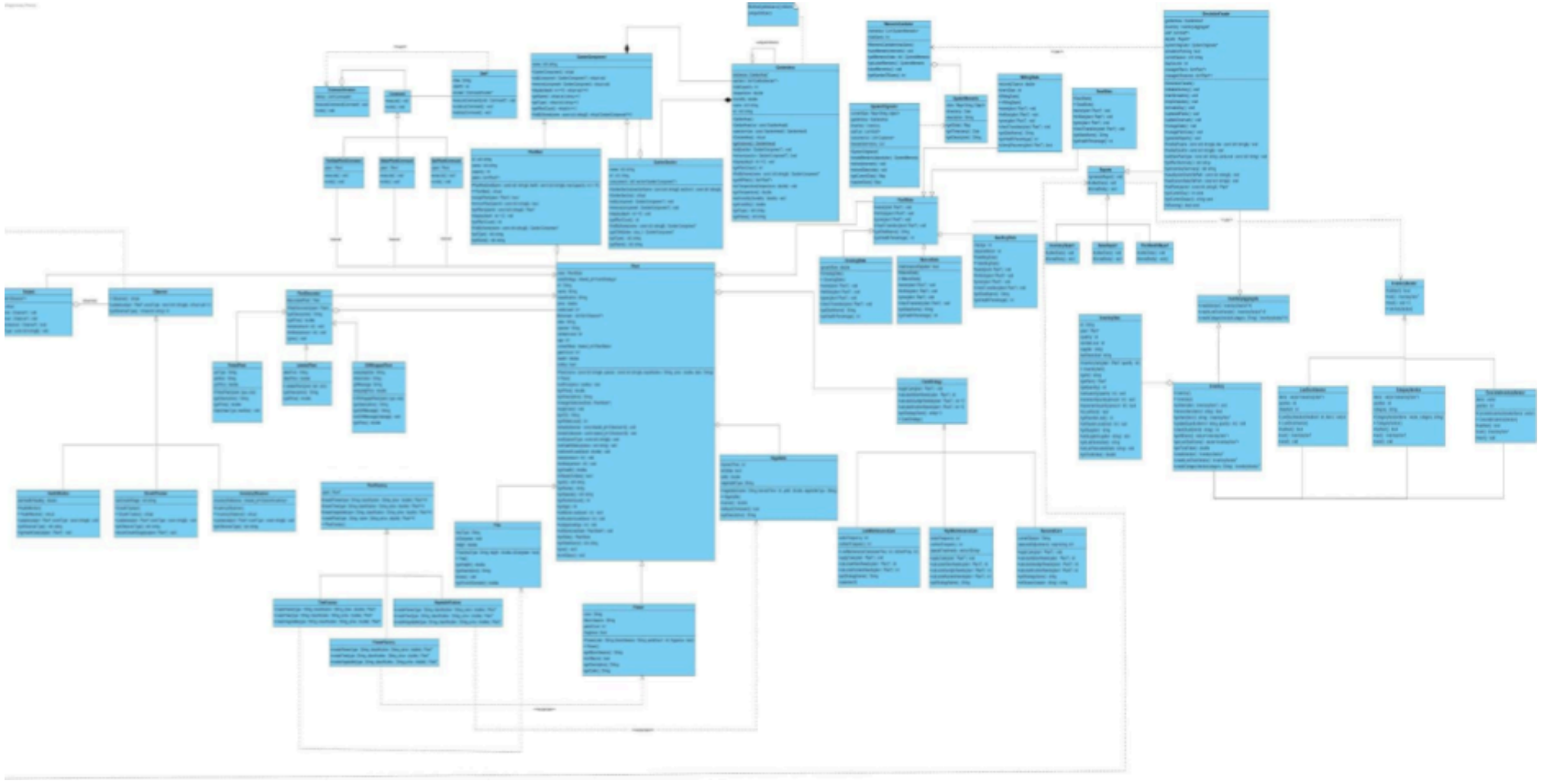
Implementation: GardenArea with private constructor and static instance
Purpose: Ensure single garden instance exists
Justification: A nursery has one physical garden area. Singleton provides global access while preventing multiple instances.
Functional Requirement: FR12 - Manage single garden area instance

Functional Requirement

Functional Requirement	Design Pattern	Implementation Class
Create plant types	Factory Method	PlantFactory
Create plant packages	Abstract Factory	PlantAbstractFactory
Track plant states	State	PlantState
Apply care strategies	Strategy	CareStrategy
Monitor Changes	Observer	Observer & Subject
Staff operations	Command	Staff
Garden organization	Composite	GardenComponent

Inventory traversal	Iterator	InventoryIterator
Plant enhancements	Decorator	PlantDecorator
Report generation	Template Method	Report
System interface	Facade	SimulationFacade
State	Memento	SystemMemento
Garden management	Singleton	GardenArea

UML Class Diagram



Implementation

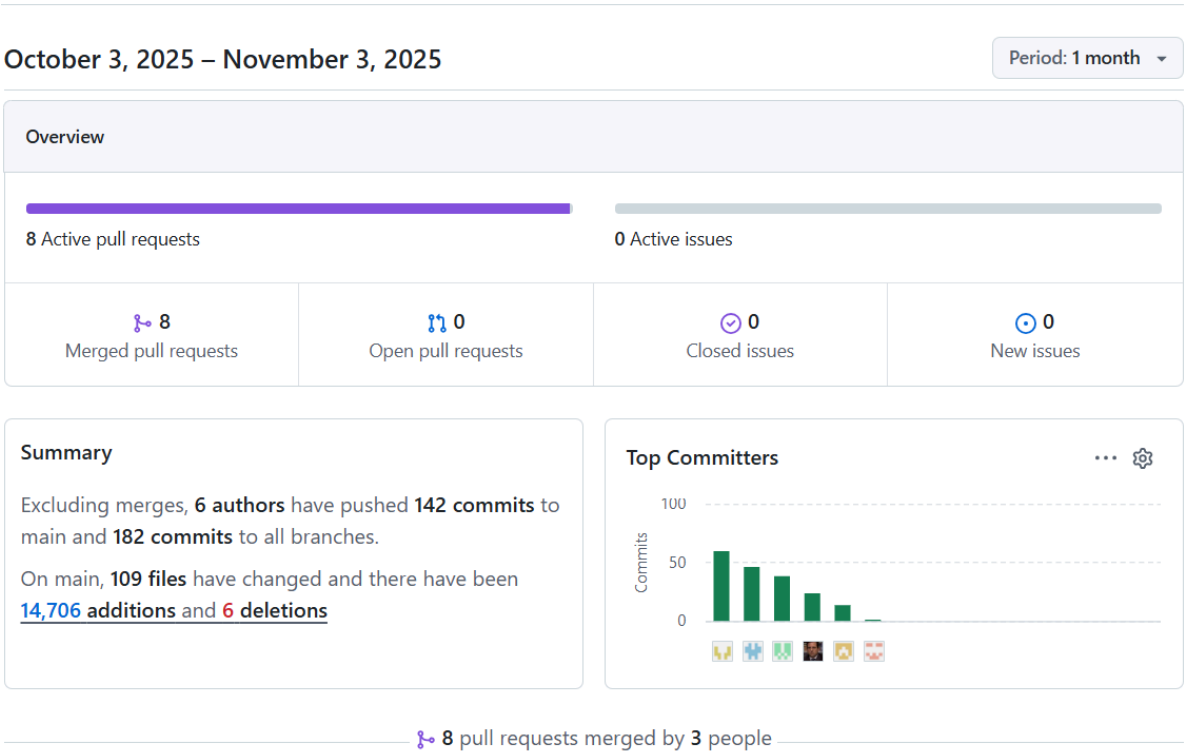
GitHub was used for the implementation of the project. We divided the design patterns among team members, and each member created their own branch in the repository to work on their respective implementations. After completing individual tasks, all branches were merged into the **development (dev)** branch for integration and further testing.

The project repository can be accessed here: **GitHub Repository Link**

A detailed record of how the design patterns were allocated and implemented by each team member is documented here: **Pattern Allocation**

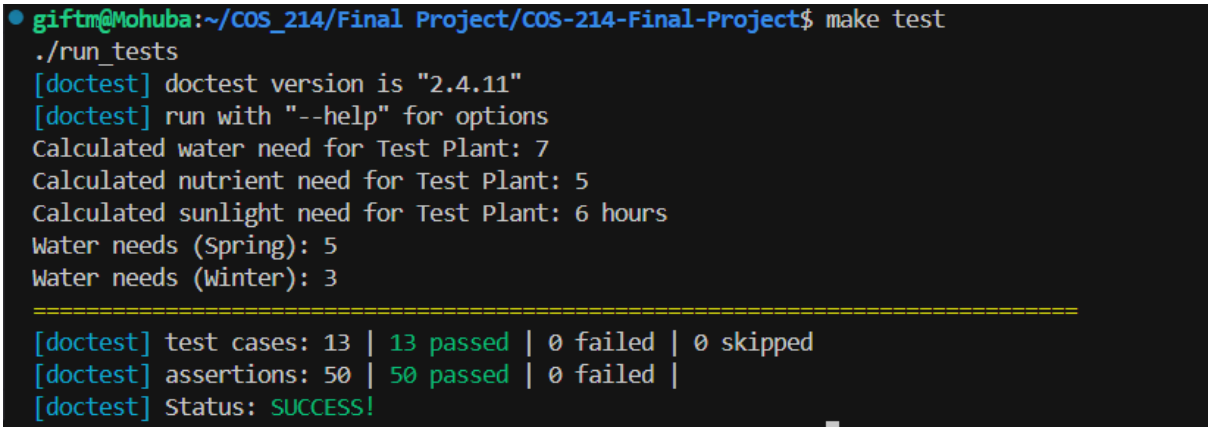
All code was documented using **Doxygen** to ensure clarity and maintainability. Unit testing was also implemented, with each team member responsible for writing a portion of the tests to validate their contributions.

Commits



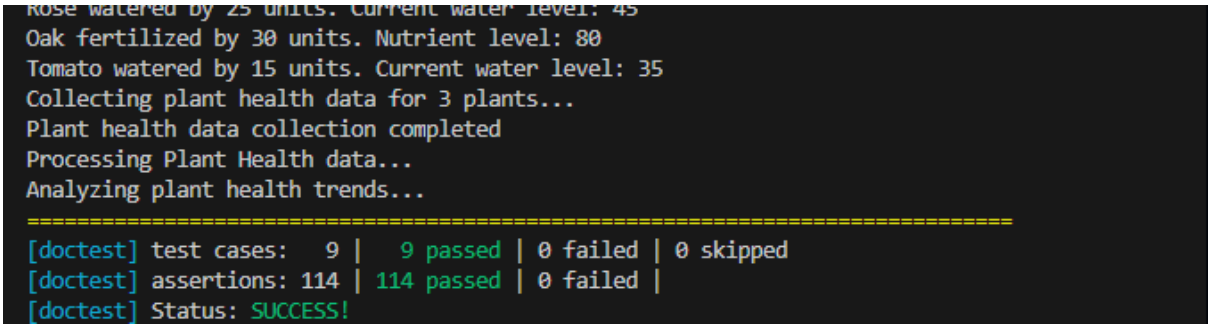
UNIT TESTING

Factory & Strategy & Iterator Unit Testing



Code for testing is found in this branch :
https://github.com/GiftMHB/COS-214-Final-Project/tree/Gift-Factory-Strategy-Iterator

AbstractFactory & Command & Template Method Unit Testing



State Unit Testing

```
root@EandJ-Chris:~/COS-214-Final-Project# make test-doctest
Wilting Wilting Plant has been wilting for 2 days
Wilting Wilting Plant has been wilting for 3 days
Wilting Wilting Plant has been wilting for 4 days
Wilting Wilting Plant has been wilting for 5 days
Wilting Wilting Plant has been wilting for 6 days
Wilting Wilting Plant has been wilting for 7 days
Wilting Wilting Plant has been wilting for 8 days
Wilting Plant has died after prolonged wilting!
Wilting Plant changed state to: Dead
Dead plant Wilting Plant cannot grow
Dead plant Wilting Plant cannot grow
Created new plant: Dead Plant (ID: PLANT-10)
Dead Plant changed state to: Dead
Cannot water dead plant Dead Plant
Created new plant: Dead Plant (ID: PLANT-11)
Dead Plant changed state to: Dead
Cannot fertilize dead plant Dead Plant
Created new plant: Dead Plant (ID: PLANT-12)
Dead Plant changed state to: Dead
Dead plant Dead Plant cannot grow
Created new plant: Resource Test (ID: PLANT-13)
Created new plant: Resource Test (ID: PLANT-14)
Created new plant: Plant 1 (ID: PLANT-15)
Created new plant: Plant 2 (ID: PLANT-16)
Seedling Plant 1 aged to 11 days
Seedling Plant 1 is transitioning to Growing state!
Plant 1 changed state to: Growing
===============================================================================
[doctest] test cases: 10 | 10 passed | 0 failed | 0 skipped
[doctest] assertions: 37 | 37 passed | 0 failed |
[doctest] Status: SUCCESS!
```

Singleton & Memento & Decorator Unit Testing

```
root@Lusandas-Laptop:/mnt/c/Users/lusmt/Downloads/c214fpdp1# ./doctesttest
[doctest] doctest version is "2.4.12"
[doctest] run with "--help" for options
===============================================================================
[doctest] test cases: 3 | 3 passed | 0 failed | 0 skipped
[doctest] assertions: 9 | 9 passed | 0 failed |
[doctest] Status: SUCCESS!
root@Lusandas-Laptop:/mnt/c/Users/lusmt/Downloads/c214fpdp1#
```

Composite & Observer & Facade Unit Testing

```
[doctest] doctest version is "2.4.12"
[doctest] run with "--help" for options
PlantBed 'Test Bed' is at full capacity!
 Initializing Plant Nursery Simulation...
 Nursery simulation shut down.
 Initializing Plant Nursery Simulation...

=== Initializing Nursery ===
 Garden structure created
 Environmental controls set
 Nursery initialization complete!

 Nursery simulation shut down.
 Initializing Plant Nursery Simulation...

=== Initializing Nursery ===
 Garden structure created
 Environmental controls set
 Nursery initialization complete!


 Simulation started! Season: Spring
 Nursery simulation shut down.
 Initializing Plant Nursery Simulation...

=== Initializing Nursery ===
 Garden structure created
 Environmental controls set
 Nursery initialization complete!
```

Composite & Observer & Facade Unit Testing continued…

```
 Simulation started! Season: Spring

=== Day 1 - Spring ===
 Staff cared for 0 plant needs
 Updated 0 plants
 0 plants ready for sale
 Observers notified of changes

=== Day 2 - Spring ===
 Staff cared for 0 plant needs
 Updated 0 plants
 0 plants ready for sale
 Observers notified of changes

=== Day 3 - Spring ===
 Staff cared for 0 plant needs
 Updated 0 plants
 0 plants ready for sale
 Observers notified of changes

=== Day 4 - Spring ===
 Staff cared for 0 plant needs
 Updated 0 plants
 0 plants ready for sale
 Observers notified of changes

=== Day 5 - Spring ===
 Staff cared for 0 plant needs
 Updated 0 plants
 0 plants ready for sale
 Observers notified of changes
```

```
Test Plant watered by 20 units. Current water level: 70
Test Plant fertilized by 15 units. Nutrient level: 65
Test Plant watered by -40 units. Current water level: 10
Doomed Plant watered by -20 units. Current water level: 30
Doomed Plant fertilized by -20 units. Nutrient level: 30
Doomed Plant watered by -20 units. Current water level: 10
Doomed Plant fertilized by -20 units. Nutrient level: 10
Doomed Plant watered by -20 units. Current water level: 0
Doomed Plant fertilized by -20 units. Nutrient level: 0
Doomed Plant watered by -20 units. Current water level: 0
Doomed Plant fertilized by -20 units. Nutrient level: 0
Doomed Plant watered by -20 units. Current water level: 0
Doomed Plant fertilized by -20 units. Nutrient level: 0
Doomed Plant watered by -20 units. Current water level: 0
Doomed Plant fertilized by -20 units. Nutrient level: 0
Doomed Plant watered by -20 units. Current water level: 0
Doomed Plant fertilized by -20 units. Nutrient level: 0
Doomed Plant watered by -20 units. Current water level: 0
Doomed Plant fertilized by -20 units. Nutrient level: 0
Doomed Plant watered by -20 units. Current water level: 0
Doomed Plant fertilized by -20 units. Nutrient level: 0
Doomed Plant watered by -20 units. Current water level: 0
Doomed Plant fertilized by -20 units. Nutrient level: 0
Sick watered by -30 units. Current water level: 20
Observed Plant watered by 10 units. Current water level: 60
Observed Plant watered by 10 units. Current water level: 70
Observed Plant watered by 10 units. Current water level: 60
Multi Observed watered by 5 units. Current water level: 55
Multi Observed watered by 5 units. Current water level: 60
Multi Observed watered by 5 units. Current water level: 65
Multi Observed watered by 5 units. Current water level: 70
Multi Observed watered by 5 units. Current water level: 75
```

The final verdict of the Composite, Observer and Facade Unit Testing:

```
Sick watered by -30 units. Current water level: 20
Observed Plant watered by 10 units. Current water level: 60
Observed Plant watered by 10 units. Current water level: 70
Observed Plant watered by 10 units. Current water level: 60
Multi Observed watered by 5 units. Current water level: 55
Multi Observed watered by 5 units. Current water level: 60
Multi Observed watered by 5 units. Current water level: 65
Multi Observed watered by 5 units. Current water level: 70
Multi Observed watered by 5 units. Current water level: 75
Multi Observed watered by 10 units. Current water level: 85
Edge Case Plant watered by 10 units. Current water level: 60
Edge Case Plant watered by 10 units. Current water level: 60
===============================================================================
[doctest] test cases:  21 |  21 passed | 0 failed | 0 skipped
[doctest] assertions: 113 | 113 passed | 0 failed |
[doctest] Status: SUCCESS!
```

The code for this testing can be found here:
https://github.com/GiftMHB/COS-214-Final-Project/tree/Tiego---Composite_Observer_Facade